

Implement Client Script & UI Policy (Incident)

Phase 8 – Project Demonstration

8.1 Purpose

The purpose of this phase is to demonstrate the complete working process of the **Implement Client Script & UI Policy (Incident)** project and show that the implemented ServiceNow configurations work according to the project requirements.

The demonstration covers the Incident form, UI Policy, UI Policy Action, Client Scripts, testing activities, and the resulting field behaviour.

8.2 Project Demonstration

The complete project demonstration should be performed in the ServiceNow **Incident Management** environment.

8.2.1 Open the Incident Module

1. Open the **ServiceNow** instance.
 2. Navigate to the **Incident** application.
 3. Open **Incident** → **All**.
 4. Open an existing Incident or create a new Incident record.
-

8.2.2 Demonstrate High Impact Control

1. Open the Incident record.
2. Set **Impact** to **1 – High**.
3. Observe the Incident form.
4. Verify that **Assignment Group** becomes mandatory.
5. Verify that the **Urgency** field becomes read-only.

This demonstrates the **High Impact Control UI Policy** and the **Urgency UI Policy Action**.

8.2.3 Demonstrate Automatic Urgency Update

1. Change the **Impact** field to **1 – High**.
2. Observe the **Urgency** field.

3. Verify that Urgency is automatically set to **High**.
4. Observe the informational message displayed by the Client Script.

This demonstrates the **onChange Client Script – Auto set urgency for high impact**.

8.2.4 Demonstrate Mandatory Enforcement

1. Keep **Impact** set to **1 – High**.
2. Leave **Assigned To** empty.
3. Attempt to submit the Incident.
4. Observe the validation message.
5. Verify that the Incident is not submitted.

This demonstrates the required validation behaviour for a High Impact Incident.

8.2.5 Demonstrate Successful Save

1. Keep **Impact** set to **1 – High**.
2. Select a valid **Assigned To** user.
3. Provide the required Incident information.
4. Submit the Incident.
5. Verify that the Incident is saved successfully.

This demonstrates successful Incident submission when the required information is provided.

8.2.6 Demonstrate Reverse Condition

1. Open the Incident record.
2. Change **Impact** from **1 – High** to **2 – Medium**.
3. Observe the fields controlled by the UI Policy.
4. Verify that the High Impact UI Policy is no longer applied.
5. Verify that the configured field behaviour returns to its normal state.

This demonstrates the **Reverse if false** behaviour of the UI Policy.

8.2.7 Demonstrate State List-Edit Blocking

1. Navigate to **Incident** → **All**.
2. Locate an Incident record.
3. Attempt to change the **State** directly from the Incident list.
4. Observe the warning message.
5. Verify that the State change is blocked.

This demonstrates the **onCellEdit Client Script – Prevent state change via list edit**.

8.2.8 Demonstrate Form-Based State Update

1. Open the Incident record.
2. Open the Incident form.
3. Change the **State** through the form.
4. Save or update the Incident.
5. Verify that the State change is accepted.

This demonstrates that valid State updates can still be performed through the Incident form.

8.3 Complete Demonstration Flow

The overall demonstration can be represented as:

Incident → **Impact = High** → **UI Policy Activation** → **Assignment Group Mandatory** → **Urgency Control** → **Automatic Urgency Update** → **Submission Validation** → **Successful Save**

For the reverse condition:

Impact = High → **Change Impact to Medium** → **UI Policy Condition False** → **Field Behaviour Reversed**

For State control:

Incident List → **State List Edit** → **onCellEdit Client Script** → **Warning** → **State Change Blocked**

For a valid State update:

Incident Form → **State Update** → **Form Submission** → **Update Successful**

8.4 Demonstration of Project Components

During the demonstration, the following implemented components should be explained:

UI Policy

High Impact Control

- Table: Incident
- Condition: Impact is **1 – High**
- Active: Yes
- Reverse if false: Yes

UI Policy Action

Urgency

- Field: Urgency
- Read-only: Yes

onChange Client Script

Auto set urgency for high impact

Automatically sets Urgency to High when Impact becomes High.

onSubmit Client Script

Prevent save if Assigned To missing

Prevents submission when Impact is High and Assigned To is empty.

onCellEdit Client Script

Prevent state change via list edit

Prevents direct State changes through Incident list editing.

8.5 Demonstration of Testing Results

The five testing activities completed in **Phase 6** should be included in the final demonstration:

Test	Demonstration Result
Mandatory Enforcement	Assignment Group becomes mandatory for High Impact.

Test	Demonstration Result
Successful Save	Incident is saved successfully when required information is provided.
Reverse Condition	UI Policy behaviour is reversed when Impact changes from High to Medium.
List Edit Blocking	Direct State change from the Incident list is blocked.
Form-Based Update	State can be successfully changed through the Incident form.

This demonstrates that the implemented configuration was not only created but also tested under the required conditions.

8.6 Demonstration Evidence

The demonstration can be supported by the screenshots already included in **Phase 7 – Project Documentation**.

The relevant evidence includes:

- High Impact Control UI Policy
- Urgency UI Policy Action
- onChange Client Script
- onSubmit Client Script
- onCellEdit Client Script
- Mandatory Enforcement test
- Successful Save test
- Reverse Condition test
- State List-Edit Blocking test
- Form-Based State Update test

8.7 Presentation Flow

The project presentation can follow this sequence:

Slide Content

- 1 Project title and team members

Slide Content

- 2 Problem statement
 - 3 User story and project objectives
 - 4 Functional and non-functional requirements
 - 5 System design and workflow
 - 6 UI Policy and UI Policy Action
 - 7 Client Scripts
 - 8 Testing activities and results
 - 9 Complete project demonstration
 - 10 Project results
 - 11 Conclusion
-

8.8 Final Demonstration Outcome

The demonstration should show that the **Implement Client Script & UI Policy (Incident)** project:

- Dynamically controls Incident fields.
- Makes **Assignment Group** mandatory for High Impact Incidents.
- Controls the **Urgency** field.
- Automatically updates Urgency when Impact becomes High.
- Validates Incident submission when Assigned To is missing.
- Allows successful submission when the required information is provided.
- Reverses UI Policy behaviour when the condition becomes false.
- Blocks direct State changes through Incident list editing.
- Allows valid State updates through the Incident form.

The demonstration provides practical evidence that the implemented ServiceNow configuration has been developed and tested according to the project requirements.

8.9 Final Submission

The final project submission should contain the completed **eight phase documents** and supporting project evidence.

Final Submission Checklist

- Phase 1 – Brainstorming & Ideation
- Phase 2 – Requirement Analysis
- Phase 3 – Project Design
- Phase 4 – Project Planning
- Phase 5 – Project Development
- Phase 6 – Project Testing
- Phase 7 – Project Documentation
- Phase 8 – Project Demonstration
- Supporting screenshots and project evidence
- Client Script source files, if required
- Organized GitHub repository

8.10 Conclusion

The **Implement Client Script & UI Policy (Incident)** project demonstrates the practical use of ServiceNow UI Policies and Client Scripts for controlling Incident form behaviour.

The final demonstration covers the complete project functionality, including dynamic field control, automatic value updates, submission validation, reverse condition behaviour, State list-edit blocking, and form-based State updates.

The completed eight-phase documentation, testing results, screenshots, and project evidence provide a structured record of the project from initial idea through development, testing, documentation, and final demonstration.